Chapter 7: Transaction Processing and Concurrency Control

Assignment Questions

1. **Explain ACID properties in detail. Explain with an example of how they ensure reliable transaction management.**
2. **Compare serial and serializable schedules with examples of each.**
3. **Define two-phase locking (2PL). Explain its role in ensuring concurrency control with examples.**
4. **What is timestamp-based concurrency control? Provide real-world examples of its application.**
5. **Discuss multi-version concurrency control (MVCC). Explain how it avoids locking overhead.**
6. **Write an SQL program to demonstrate conflict serializability using a small dataset.**

Chapter 8. Transaction Processing and Concurrency Control (4 Hrs)

1. Transactions and its Properties

Definition of a Transaction

- A **transaction** is a sequence of one or more operations (such as read, write, update, or delete) performed on a database, treated as a single logical unit of work.
- Transactions ensure that database integrity is maintained, even in the presence of **concurrent access** by multiple users or **system failures**.

Why Transactions Are Important

1. **Consistency in Multi-User Environments**:
    - Prevents issues like dirty reads, lost updates, or inconsistent data when multiple users interact with the database simultaneously.
2. **Error Recovery**:
    - Ensures the database can recover to a consistent state in the event of failures like crashes or power outages.
3. **Adherence to Business Rules**:
    - Transactions enforce business logic, such as ensuring that bank balances never go negative.

Properties of Transactions (ACID Properties)

1. Atomicity

- **Definition**:
    - Atomicity ensures that a transaction is treated as an indivisible unit. Either all the operations in the transaction are completed successfully, or none of them are applied to the database.
- **How It Works**:
    - If any operation within a transaction fails, all changes made by the transaction are rolled back.
- **Example**:
    - **Bank Transfer**:

        - A transaction debits $500 from Account A and credits it to Account B. If the debit succeeds but the credit fails, the transaction rolls back, and neither account is affected.

---

2. Consistency

- **Definition**:
    - Consistency ensures that a transaction moves the database from one valid state to another while adhering to all integrity constraints.
- **How It Works**:
    - Transactions must preserve database rules, such as foreign key relationships, unique constraints, or domain constraints.
- **Example**:
    - **Inventory Management**:
        - If an order reduces the inventory of an item, the total inventory count should remain accurate after the transaction. A violation (e.g., negative inventory) would break consistency.

---

3. Isolation

- **Definition**:
    - Isolation ensures that transactions are executed as though they are the only transaction running in the database. The intermediate state of a transaction is not visible to other transactions.
- **How It Works**:
    - Isolation is enforced through concurrency control mechanisms like locks, timestamps, or multi-version protocols.
- **Example**:
    - **Simultaneous Withdrawals**:
        - Two users attempt to withdraw $500 from the same account with $800 available:
            - Without isolation: Both transactions could read $800 and proceed, leading to overdrawn funds.
            - With isolation: The first transaction locks the account balance until it commits or aborts.

---

4. Durability

- **Definition**:
    - Durability ensures that once a transaction is committed, its changes are permanent and survive any subsequent failures (e.g., power outages or crashes).
- **How It Works**:
    - DBMS writes changes to persistent storage (disk) before considering a transaction committed.
- **Example**:
    - **Airline Booking**:
        - A successful booking transaction guarantees that the ticket remains booked even if the system crashes immediately afterward.

Example of a Transaction

Scenario: Transferring $1000 from Account A to Account B.

Operations in the Transaction:

1. Read balance of Account A.
2. Deduct $1000 from Account A.
3. Read balance of Account B.
4. Add $1000 to Account B.
5. Commit the changes.

Key Features:

- **Atomicity**: If any step fails (e.g., insufficient funds), all changes are rolled back.
- **Consistency**: The total balance of Account A and Account B remains unchanged.
- **Isolation**: Other transactions cannot access the intermediate states of this transaction.
- **Durability**: After the commit, the changes are permanent, even if a failure occurs.

Challenges in Transaction Management

1. **Concurrency Issues**:

- Problems like dirty reads, lost updates, or uncommitted data can arise without proper isolation.

2. **System Failures**:
    - Crashes can interrupt transactions, requiring recovery mechanisms like undo/redo logging.
3. **Performance Overhead**:
    - Ensuring ACID properties often introduces latency, especially in high-transaction environments.

Real-World Applications of Transactions

1. **Banking Systems**:
    - Transactions ensure that fund transfers, deposits, and withdrawals adhere to ACID properties.
    - Example: A failed ATM withdrawal doesn't deduct funds from the account.
2. **E-Commerce**:
    - Guarantees consistency in order processing and inventory management.
    - Example: Deducting inventory and charging the customer occur as part of a single transaction.
3. **Reservation Systems**:
    - Prevents overbooking of tickets or rooms during concurrent reservations.
    - Example: Two customers cannot book the same seat on a flight.

Summary

- **Transactions** are essential for maintaining data integrity and consistency in database systems, especially in multi-user and failure-prone environments.
- The **ACID properties**:
    1. **Atomicity** ensures that a transaction is an all-or-nothing operation.
    2. **Consistency** ensures adherence to database rules and constraints.
    3. **Isolation** prevents transactions from interfering with each other.
    4. **Durability** ensures that committed changes are permanent.

These properties form the foundation of reliable and consistent database management.

2. Serial and Serializable Schedules

Serial Schedule

Definition

- A **serial schedule** is a sequence of transactions where only one transaction is executed at a time, and the next transaction begins only after the current transaction is completed. There is no overlapping or interleaving of operations.

Advantages

1. **Data Consistency**:
    - Since transactions are executed sequentially, no concurrent modifications can occur, ensuring data integrity.
2. **Simpler Implementation**:
    - No need for complex concurrency control mechanisms.

Disadvantages

1. **Poor Performance**:
    - Lack of parallelism leads to underutilization of system resources, especially in multi-user environments.
2. **Scalability Issues**:
    - Inefficient for systems with high transaction volumes as users must wait for previous transactions to complete.

Example

Serial Schedule:

- **T1** completes all its operations before **T2** starts.

  T1: Read(A), Update(A), Commit

  T2: Read(B), Update(B), Commit

Execution:

- Transaction **T1**:
    - Reads and modifies data item A.
    - Commits changes.
- Transaction **T2**:
    - Reads and modifies data item B.
    - Commits changes.

Serializable Schedule

Definition

- A **serializable schedule** allows the concurrent execution of transactions but ensures that the final result is equivalent to some serial execution of those transactions.

Advantages

1. **Improved Performance**:
    - Allows parallel execution of transactions, reducing wait times and improving resource utilization.
2. **Ensures Consistency**:
    - Maintains the same results as a serial schedule, ensuring data integrity.

Disadvantages

1. **Complex Implementation**:
    - Requires concurrency control mechanisms like locking, timestamping, or validation to maintain serializability.
2. **Overhead**:
    - Detecting and ensuring serializability may introduce additional computational costs.

Types of Serializable Schedules

1. **Conflict Serializability**
    - **Definition**:

        - A schedule is conflict-serializable if it can be transformed into a serial schedule by swapping non-conflicting operations.
    - **Conflict**:
        - Two operations conflict if:
            1. They access the same data item.
            2. At least one of them is a write operation.
    - **Example**:
        - Schedule:

          T1: Read(A), Write(A)

          T2: Read(A), Write(A)

            - This schedule is **not conflict-serializable** because the operations on A conflict.
    - **Advantages**:
        - Conflict serializability is easy to test using dependency graphs.
    - **Disadvantages**:
        - It is stricter and may disallow some valid schedules.

---

2. **View Serializability**
    - **Definition**:
        - A schedule is view-serializable if it produces the same final result as a serial schedule but may not be conflict-serializable.
    - **Example**:
        - Two schedules that write different data items but produce the same final database state are view-serializable.
    - **Advantages**:
        - Allows more concurrent schedules than conflict serializability.
    - **Disadvantages**:
        - More difficult to test compared to conflict serializability.

Example of Serializable Schedule

Transactions:

- **T1: Read(A), Update(A), Commit**
- **T2: Read(B), Update(B), Commit**

Serializable Schedule:

T1: Read(A)

T2: Read(B)

T2: Update(B)

T1: Update(A)

Commit(T1)

Commit(T2)

Explanation:

- This schedule executes parts of **T1** and **T2** concurrently, but the final result is equivalent to executing **T1** and **T2** sequentially.

Key Differences: Serial vs Serializable Schedules

Aspect	Serial Schedule	Serializable Schedule
Execution	Transactions execute one after another.	Transactions execute concurrently.
Performance	Poor due to lack of parallelism.	High due to concurrent execution.
Complexity	Simple to implement.	Requires concurrency control mechanisms.
Real-World Feasibility	Impractical for multi-user systems.	Suitable for multi-user and distributed systems.
Data Integrity	Always ensures consistency.	Ensures consistency equivalent to a serial schedule.

Applications of Serializable Schedules

1. **Banking Systems**:
    - Ensure multiple transactions (e.g., fund transfers) produce consistent results while processing concurrently.
2. **E-commerce Platforms**:
    - Allows concurrent order placements and inventory updates without conflicts.
3. **Reservation Systems**:

- Ensures that users do not book the same resource (e.g., a seat or room) simultaneously.

Summary

- **Serial Schedule**:
  - Executes transactions one after another, ensuring consistency but with poor performance in multi-user environments.
- **Serializable Schedule**:
  - Allows concurrent execution of transactions while maintaining the same results as a serial schedule, improving performance and resource utilization.
- **Types of Serializable Schedules**:

1. **Conflict Serializability**:
   - Focuses on detecting and resolving conflicts between transactions.
2. **View Serializability**:
   - Ensures the same final database state as a serial schedule, allowing greater concurrency.

3. Conflict Serializability

Definition

- A schedule is **conflict-serializable** if it can be transformed into a serial schedule (i.e., transactions are executed one after another) by **swapping non-conflicting operations** without changing the outcome of the transactions.

Conflicting Operations

Two operations are said to conflict if:

1. **They access the same data item**, and
2. **At least one of the operations is a write**.

Types of Conflicts

1. **Write-Write Conflict**:

- Two transactions try to write to the same data item.
- Example:

  T1: Write(A)

  T2: Write(A)

2. **Read-Write Conflict**:
   - One transaction reads a data item while another transaction writes to the same data item.
   - Example:

     T1: Read(A)

     T2: Write(A)
3. **Write-Read Conflict**:
   - One transaction writes to a data item while another transaction reads the same data item.
   - Example:

     T1: Write(A)

     T2: Read(A)

Example of a Conflict-Serializable Schedule

Schedule:

T1: Read(A), Write(A)

T2: Read(B), Write(B)

Conflict Detection:

- No conflict occurs between T1 and T2 as they access different data items (A and B).

Result:

- This schedule is conflict-serializable as it can be executed in the order T1 → T2 or T2 → T1.

Example of a Non-Conflict-Serializable Schedule

Schedule:

T1: Read(A), Write(A)

T2: Read(A), Write(A)

Conflict Detection:

- There is a conflict on the data item A:
    - T1's **Write(A)** conflicts with T2's **Read(A)** and **Write(A)**.

Result:

- This schedule is **not conflict-serializable** because the operations cannot be rearranged into a serial order.

Advantages of Conflict Serializability

1. **Guarantees Consistency**:
    - Ensures that the database remains consistent by enforcing serializability.
2. **Conflict Detection**:
    - Conflicts can be identified and managed using dependency graphs or concurrency control protocols.
3. **Simple Verification**:
    - Easy to test using conflict graphs (precedence graphs).

Disadvantages of Conflict Serializability

1. **Restrictive**:
    - Some valid schedules that are not conflict-serializable are disallowed.
2. **Overhead**:
    - Requires additional resources for conflict detection and resolution.

---

4. Concurrency Control Techniques

Concurrency control ensures that multiple transactions can execute concurrently without leading to data inconsistency.

4.1 Locking

Definition

Locking is a concurrency control mechanism used to restrict access to data items during concurrent transactions to ensure data consistency and integrity.

Types of Locks

1. **Shared Lock (S)**:
    - Allows multiple transactions to **read** a data item simultaneously but prevents write operations.
    - Example:
        - Two transactions reading the balance of an account can hold a shared lock on the balance.
2. **Exclusive Lock (X)**:
    - Allows a transaction to both **read and write** a data item.
    - Prevents other transactions from accessing the data item (neither read nor write).
    - Example:
        - A transaction updating an account balance acquires an exclusive lock, ensuring no other transaction can read or write to the same account during the update.

Two-Phase Locking (2PL)

1. **Growing Phase**:
    - A transaction acquires all the locks it needs but cannot release any lock during this phase.
2. **Shrinking Phase**:
    - A transaction releases locks but cannot acquire new locks.

Strict Two-Phase Locking (Strict 2PL)

- Ensures that a transaction releases all locks only **after committing**.
- Provides better isolation and prevents cascading rollbacks.

Advantages

1. **Prevents Data Inconsistency**:
    - Ensures that conflicting transactions do not access the same data item simultaneously.

2. **Ensures Serializability**:
    - Transactions execute as if they were performed serially.

Disadvantages

1. **Deadlocks**:
    - Occur when two or more transactions wait indefinitely for resources held by each other.
    - Example:
        - T1 locks A, T2 locks B. T1 waits for B, and T2 waits for A.
2. **Performance Overhead**:
    - Increased resource contention and waiting times can reduce throughput in highly concurrent systems.

4.2 Timestamp-Based Protocols

Definition

Timestamp-based concurrency control assigns a unique **timestamp** to each transaction to establish a global order of execution, ensuring serializability.

Steps

1. **Transaction Priority**:
    - Transactions are ordered based on their timestamps. Older transactions (with smaller timestamps) get higher priority.
2. **Validation**:
    - If a transaction attempts to perform an operation that violates the timestamp order, it is aborted and restarted with a new timestamp.

Advantages

1. **Ensures Serializability**:
    - Transactions execute in a globally consistent order.
2. **Avoids Deadlocks**:

- No need for locks, as conflicts are resolved through aborts.

Disadvantages

1. **Frequent Transaction Restarts**:
   - High contention environments may result in frequent restarts, reducing performance.
2. **Overhead**:
   - Managing timestamps for every transaction adds complexity.

4.3 Validation-Based Protocols

Definition

Validation-based protocols divide a transaction into **three phases** to ensure that no conflicts occur with other transactions before changes are applied to the database.

Phases

1. **Read Phase**:
   - The transaction reads data items and performs computations without modifying the database.
2. **Validation Phase**:
   - Before committing, the transaction checks whether its changes conflict with other concurrent transactions.
3. **Write Phase**:
   - If validation is successful, the transaction writes its changes to the database.

Advantages

1. **High Performance**:
   - Works well in environments with minimal transaction conflicts.
2. **Avoids Locking**:
   - No locks are required, reducing resource contention.

Disadvantages

1. **Validation Failures**:
    - Transactions that fail the validation phase are aborted and restarted, leading to wasted work.
2. **Not Suitable for High-Conflict Environments**:
    - Frequent validation failures can degrade performance.

4.4 Multi-Version Concurrency Control (MVCC)

Definition

MVCC is a concurrency control technique that maintains **multiple versions** of data items, allowing transactions to read and write concurrently without conflicts.

Steps

1. **Read Operations**:
    - A transaction reads the version of the data item that was valid when the transaction started.
2. **Write Operations**:
    - A transaction creates a new version of the data item without overwriting existing versions.

Advantages

1. **High Concurrency**:
    - Transactions can proceed without waiting for locks, improving throughput.
2. **Avoids Locking Overhead**:
    - Eliminates contention caused by locks.

Disadvantages

1. **Increased Storage Requirements**:
    - Multiple versions of data items require additional storage space.
2. **Complex Version Management**:
    - Maintaining and cleaning up old versions adds overhead.

Advantages of Concurrency Control

1. **Ensures Data Consistency**:
    - Prevents anomalies like dirty reads, lost updates, or uncommitted data.
2. **Maximizes Resource Utilization**:
    - Enables parallel execution of transactions, improving performance.
3. **Prevents Data Corruption**:
    - Ensures that transactions do not interfere with each other.

Disadvantages of Concurrency Control

1. **Performance Overhead**:
    - Techniques like locking and validation add extra processing and memory usage.
2. **Increased Complexity**:
    - Implementing and managing concurrency control protocols is challenging.
3. **Deadlocks and Restarts**:
    - Deadlocks in locking-based protocols and restarts in timestamp/validation protocols can degrade user experience.

Real-World Examples of Concurrency Control

1. **Banking Systems**:
    - Ensures simultaneous transactions (e.g., fund transfers) do not lead to inconsistencies or negative balances.
    - Example: MVCC can allow users to view account balances while updates are in progress.
2. **E-Commerce Platforms**:
    - Prevents anomalies like double booking of items or incorrect inventory updates during high traffic.
3. **Reservation Systems**:
    - Ensures that two users cannot book the same seat or hotel room at the same time.

Key Takeaways

- **Locking** ensures serializability through resource control but may lead to deadlocks.
- **Timestamp-based protocols** enforce global transaction order but may restart conflicting transactions.
- **Validation-based protocols** excel in low-conflict environments by validating transactions before commit.
- **MVCC** provides high concurrency by allowing simultaneous reads and writes through multiple data versions.

Each technique has trade-offs, and the choice depends on the specific requirements of the system, such as the expected workload and acceptable level of performance overhead.